OpenZeppelin | security

DollarStore Update Audit



November 4, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 6

Security Model and Trust Assumptions 7

Privileged Roles 7

Medium Severity 8

M-01 Any User Can Indefinitely Block Retirement of a Winding-Down Spoke 8

M-02 haircutEscrow Repeatedly Reduces the Queues of Relisted Assets 9

M-03 A Frozen Wallet Can Block Other Users' Fills and Deposits Using Parked Positions 10

Low Severity 11

L-01 haircutEscrow Can Be Blocked by a Few Hundred Parked Positions 11

L-02 Impaired Escrow Can Be Relisted Before Being Haircut 12

Notes & Additional Information 13

N-01 Outdated Guardian Permissions in SECURITY.md 13

N-02 Floating Pragma 13

N-03 Misleading Documentation of processQueue Return Value 14

N-04 getMinimumOrderSize Contradicts Its Documentation for the Parked Line 14

Conclusion 15

Appendix 16

Issue Classification 16

Summary

Type	DeFi
Timeline	From 2026-09-25 To 2026-09-25
Languages	Solidity

Total Issues	9 (8 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	3 (3 resolved)
Low Severity Issues	2 (1 resolved)
Notes & Additional Information	4 (4 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a diff audit of pull request #27 and pull request #28 in the `coasify/dollar` repository at the `1677698` commit, against the `913a800` base commit.

In scope were the changes made to the following files:

```
contracts
└── src
    ├── DollarStore.sol
    ├── interfaces
    │   └── IDollarStore.sol
    └── libraries
        ├── SpokeLifecycleLib.sol
        ├── SpokeShareLib.sol
        └── SwapRouteLib.sol
```

Following the initial audit, OpenZeppelin reviewed pull request #32 and pull request #33, which the DollarStore team submitted in response to issues M-01 and M-02 of this report, at the `47447f1` commit against the `93e45af` base commit.

In scope were the changes made to the following files:

```
contracts
└── src
    ├── DollarStore.sol
    ├── interfaces
    │   └── IDollarStore.sol
    ├── libraries
    │   ├── QueueLib.sol
    │   ├── SpokeAdminLib.sol
    │   ├── SpokeLifecycleLib.sol
    │   ├── SpokeShareLib.sol
    │   └── SwapRouteLib.sol
    └── storage
        ├── CoreStorage.sol
        ├── QueueStorage.sol
        └── RegistryStorage.sol
```

The contracts were compiled with Solidity 0.8.24 against OpenZeppelin Contracts v5.6.1 (`5fd1781`) and OpenZeppelin Contracts Upgradeable v5.6.1 (`7bf4727`). These dependencies were not in scope and were assumed to behave as documented.

Update: *All resolutions and the final state of the audited codebase mentioned in this report are contained at commit [eb29a07](#).*

System Overview

DollarStore is an upgradeable hub-and-spoke exchange for listed stablecoins. Hub assets back the soulbound `DLRS` receipt token, while each spoke pool pairs a single spoke asset with a DLRS-side reserve and is owned by its liquidity providers through receipt shares. The audited changes modify the protocol in four ways:

- **Removal of guardian queue cancellation (pull request #27):** The guardian-gated `adminCancelQueue` function, which allowed the Guardian to force-cancel any queue position and return the escrow to its owner, is removed. A queue position can now only be closed by being filled or by its owner through `cancelQueue`.
- **Recovery of fully impaired spokes (pull request #28):** A spoke whose spoke-asset and DLRS-side reserves are both zero while receipt shares remain outstanding is treated as fully impaired, and deposits and escrow-to-share conversions into it now revert. `removePool` no longer requires outstanding shares to be zero, so governance can retire such a pool once its queues are empty, leaving the worthless shares recorded under the killed `poolId`. The asset can then be relisted through `createSpoke` under a fresh `poolId`, and killed pools can no longer be unpaused or removed again.
- **Parking of unpayable queue positions (pull request #32):** When the transfer that pays a queued position fails, the position is no longer removed and its escrow converted into `DLRS` or receipt shares. Instead, it is moved to a per-asset parked line, where it cannot be filled and can only be cancelled by its owner for the escrowed asset once the transfer succeeds. `cancelQueue` attempts the transfer first and parks the position on failure, `removePool` ignores parked escrow, and `haircutEscrow` also walks the parked line.
- **Pool-scoped queue keys (pull request #33):** Directed queues are keyed by the pool they belong to as well as by the asset pair, so a relisted asset starts with empty queues and the lines of a killed pool are unreachable from its replacement. Consequently, `removePool` no longer requires the spoke asset's queues to be empty. Leftover positions stay in the killed pool's lines, cannot be filled, and remain cancellable by their owners, while `haircutEscrow` walks the lines of every pool that has listed the asset exactly once.

Security Model and Trust Assumptions

- Retiring a pool is irreversible. Governance is trusted to call `removePool` only once an impairment is final, as tokens recovered for a killed pool are not credited to its former liquidity providers and are instead treated as excess that the Governor can sweep through `rescueTokens`.
- Following pull request #33, retiring a spoke no longer requires its queues to be empty. Positions left in a killed pool's lines, as well as parked positions, never expire: they can only be cancelled by their owners, are not reported by the pair-based views, and must be covered by the position budget of any subsequent `haircutEscrow` call.
- Following pull request #32, a failed escrow transfer parks the position instead of converting its escrow. Parked escrow leaves the protocol only as the deposited asset, once its issuer allows the transfer, so a temporary freeze or an issuer-wide pause leaves a position parked until its owner cancels and re-queues. The parked line is not capped, and the Guardian is expected to pause deposits of the affected asset to stop it from growing.
- Settling a queued position only involves the issuer of the asset being paid out, so a wallet that is frozen on its escrowed asset after queueing can still be settled into the asset it requested.
- The reviewed version changes the queue key format and is intended to be deployed fresh rather than applied as an upgrade to a proxy that holds queued positions.

Privileged Roles

The privileged roles are unchanged from the previous audit, except that the Guardian can no longer cancel queue positions. In the context of the audited changes:

- **Governor** retires spokes through `removePool`, which no longer requires their queues to be empty, and relists their assets through `createSpoke`.
- **Guardian** pauses and unpauses pools and asset deposits, but can no longer unpause a killed pool, and applies escrow haircuts through `haircutEscrow`, which now also reach parked positions and escrow left in killed pools.

Medium Severity

M-01 Any User Can Indefinitely Block Retirement of a Winding-Down Spoke

The `removePool` function retires a spoke once its spoke reserves and `dlrsReserve` are zero and no queue on a route touching the spoke asset holds any depth. Following pull request #28, it is also the documented recovery path for a fully impaired spoke: governance kills the pool and relists the asset under a fresh `poolId` through `createSpoke`. A queue position can only leave the queue by being filled or by its owner calling `cancelQueue`, since pull request #27 removes the guardian-gated `adminCancelQueue` function.

However, once governance calls `windDownSpoke`, a hub-to-spoke queue position can no longer be cleared without the cooperation of its owner. Hub-to-spoke swaps remain enabled during wind-down, and `enqueueRemainder` exempts the first position of an empty queue from the minimum order size, so any user can escrow a single base unit of a hub asset against an empty spoke. The position cannot be filled from reserves, as spoke `deposit` calls revert for any pool that is not `Active`. It also cannot be matched peer-to-peer, as `validateRoute` rejects spoke-to-hub swaps for a winding-down pool, and no function returns a pool from `WindingDown` to `Active`. As a result, `removePool` reverts with `PoolNotEmpty` for as long as the owner declines to cancel, the asset remains listed, and `createSpoke` cannot relist it. This applies to any winding-down spoke, including an unimpaired one with no outstanding shares. The attacker's cost is one base unit of escrow, which remains refundable at any time, and the same outcome arises without malice from any abandoned position. This reopens the liveness gap identified in M-01 of the March 2026 DollarStore audit, which was resolved by introducing `adminCancelQueue` as the manual eviction path.

Consider reintroducing a narrowly scoped eviction path, restricted to spokes that are winding down or paused, that cancels remaining positions and returns the escrow to their owners through the existing `cancelPosition` flow. Alternatively, consider rejecting new queue positions on routes touching a winding-down spoke and allowing `removePool` to refund residual escrow in bounded batches.

Update: *Resolved in pull request #33 at commit 47447f1, together with pull request #32 at commit 7df0e70. Queue keys are now scoped by pool, so* `removePool` *no longer requires the*

spoke asset's queues to be empty. Positions left in a killed pool's lines can never be filled and remain cancellable by their owners; no eviction path was added.

The team stated:

> *Fixed. Queue keys now include the pool id, so removePool no longer requires empty queues. Leftover positions can't be filled and stay cancellable by their owner. We chose not to add an eviction path: only the owner can cancel a position.*

M-02 `haircutEscrow` Repeatedly Reduces the Queues of Relisted Assets

The `haircutEscrow` function allocates an escrow shortfall pro-rata across every queue offering the impaired asset. It iterates over the `assets` array of every pool and calls `_haircutQueue` for each wanted asset, which scales each position by `newAmt = amt * targetEscrow / escrow`, where `escrow` is the total escrow snapshotted before the loop.

However, `removePool` keeps a killed pool in the `pools` array with its `assets` array intact, and `createSpoke` pushes the same asset into a new pool when it is relisted. Since queue keys are derived from token addresses rather than pool IDs, the loop visits the same `(offerAsset, wantAsset)` queue once for the tombstone and once for the replacement pool, applying the same ratio each time. Each position therefore retains `amt * (targetEscrow / escrow)^k`, where `k` is the number of pools that have listed the wanted asset, instead of `amt * targetEscrow / escrow`. For example, with 100 USDC escrowed against a relisted spoke asset and only 50 USDC remaining in the contract, the position is reduced to 25 USDC rather than 50 USDC. The remaining 25 USDC becomes unaccounted excess that `rescueTokens` can sweep, and duplicate visits also consume the `maxPositions` budget, causing a correctly sized call to revert with `HaircutBudgetExceeded`. As a result, queue owners lose a share of their surviving escrow beyond the actual shortfall. Although this behavior predates the audited changes, pull request #28 establishes relisting as the prescribed recovery path for impaired spokes, which makes this state expected rather than exceptional.

Consider skipping pools with `Killed` status when iterating in `haircutEscrow`, or iterating over each currently listed asset exactly once. Clearing the `assets` array of killed pools is not advisable, as `redeemSpoke` and several view functions still read it.

Update: *Resolved in pull request #33 at commit 47447f1. A removed pool and its relisted replacement now have distinct queue lines, and `haircutEscrow` walks each line exactly once.*

The team stated:

> *Fixed. Queue keys now include the pool id, so a removed spoke and its relisted replacement have separate lines, and haircutEscrow visits each line once.*

M-03 A Frozen Wallet Can Block Other Users' Fills and Deposits Using Parked Positions

When a payout fails, `park` moves the position to the parked line but leaves its id in `userPositions[owner]`. Every full fill, cancel or zeroing haircut then calls `_removeUserPosition`, which scans the owner's whole array.

A frozen wallet can use the cycle described in the "Parked line has no cap" known limit of pull request #32 to park thousands of 1-unit entries for gas alone. It then opens a 1-unit position at the head of an empty line. Any spoke deposit, swap or `haircutEscrow` that fills that position has to scan the entire array, so it exceeds the per-transaction gas limit. Only the attacker can cancel the position, relisting the spoke does not stop it, and hub-to-hub lines need an upgrade. No funds are lost, but this contradicts the known-limit note that "nothing else is affected".

Consider removing ids from `userPositions` in O(1) by storing each position's index, or keeping parked ids out of the array.

Update: *Resolved in pull request #40 at commit 3726bf9. Each position now stores its index in `userPositions`, so removal is a constant-time swap-and-pop.*

The team stated:

> *Fixed. Each position now stores its index in its owner's position list, so removing a position is constant-time and no longer depends on how many other positions the owner holds. Parked positions stay in the list and remain visible through getUserQueuePositions. Our documented known limit is corrected.*

Low Severity

L-01 `haircutEscrow` Can Be Blocked by a Few Hundred Parked Positions

The `haircutEscrow` function walks every line holding escrow of an asset, including the parked line introduced in pull request #32, within a single transaction, and reverts once its budget is exhausted. The known limits of pull request #32 acknowledge that the parked line has no cap, and estimate that "a few thousand entries" put the haircut beyond a block at about 26k gas per entry. However, this estimate is based on warm storage, the binding constraint on Ethereum mainnet is the per-transaction gas cap introduced by EIP-7825 rather than the block gas limit, and each parked 1-unit position is reduced to zero and removed through `_removeUserPosition`, whose linear scan makes the cost quadratic when a single owner holds all entries. As a result, around 230 parked positions held by one wallet, or around 320 held by distinct wallets, are sufficient to make `haircutEscrow` unexecutable for the parked asset, an order of magnitude fewer than documented.

Consider allowing `haircutEscrow` to resume across multiple transactions, for example by storing a cursor and a fixed haircut ratio, or prioritizing the planned per-asset impairment index so that the cost of a haircut no longer depends on the length of the parked line. In addition, consider updating the known limit in pull request #32 to reflect the corrected threshold.

Update: *Acknowledged, not resolved. Pull request #40 removes the quadratic cost of removing one owner's parked positions, which raises the number of parked positions needed to block `haircutEscrow` to roughly 340, but the linear walk over the parked line remains. The team plans a per-asset impairment index applied when a position is paid or cancelled, so that the cost of a haircut no longer depends on the length of the parked line.*

The team stated:

> *Acknowledged. We reproduced the corrected threshold: about 320 to 355 one-unit parked positions, depending on how many wallets hold them, put haircutEscrow for that asset over the per-transaction gas limit. Our documented known limit is updated with these figures.*
>
> *Reaching that state requires all of the following: a wallet frozen by the issuer of the asset it asks for and still able to send the asset it offers; a route with no liquidity to fill its orders and an empty line, since an order that can fill pays the frozen wallet and reverts;*

> *about 320 swaps from a helper wallet, made before the pool is paused; and a later loss of the offered asset from the contract, which is the only time a haircut is used. No funds are taken from anyone. The effect is that such a loss falls on reserves and not on queued positions, and the attacker gains only if it also holds queued escrow in that asset.*
>
> *Given how narrow these conditions are, we are not changing the haircut in this release. The planned fix is a per-asset impairment index applied when a position is paid or cancelled, which makes the haircut a single write regardless of the parked line's length. It will ship as its own change with its own review. It can be added by upgrade and is designed to need no migration of existing positions.*

L-02 Impaired Escrow Can Be Relisted Before Being Haircut

Since pull request #33, escrow survives `removePool`, and the recovery path in `SECURITY.md` relists the asset through `createSpoke`. However, neither `createSpoke` nor `addHubAsset` checks that the remaining escrow is backed by the contract's balance, and nothing requires `haircutEscrow` to run first.

If an asset is relisted while its escrow is impaired, the old generation's owners can `cancelQueue` at full value against the new pool's deposits, leaving the new LPs (or DLRS backing, for a hub relist) to absorb the loss. A later `haircutEscrow` resolves to the live pool, so it requires pausing it and cuts the new generation's escrow for a loss that predates it.

Consider requiring in `createSpoke` and `addHubAsset` that the contract's balance covers `totalEscrowedByAsset`. Alternatively, consider documenting that `haircutEscrow` must precede any relist of an asset with remaining escrow.

Update: *Resolved in pull request #41 at commit 23d8e36. `createSpoke` and `addHubAsset` now revert with `EscrowImpaired` while the contract holds less of the asset than its outstanding escrow.*

The team stated:

> *Fixed. createSpoke and addHubAsset now revert while the contract holds less of an asset than its queue escrow. This applies only if the contract has lost some of an asset, for example through its issuer freezing and seizing the protocol's balance, which we consider unlikely. In that case the positions a removed pool left behind must be written down with haircutEscrow before the asset can be listed again, so a new pool's deposits*

> *can no longer pay a loss that predates them. Our documented recovery steps state the order.*

Notes & Additional Information

N-01 Outdated Guardian Permissions in `SECURITY.md`

The roles table in `SECURITY.md` still lists "cancel queue positions" among the Guardian's powers, although pull request #27 removes `adminCancelQueue` and its entry from the Guardian-gated function list.

Consider updating the roles table to reflect the Guardian's current permissions.

Update: *Resolved in commit cbcbdbb of pull request #27.*

The team stated:

> *Fixed. Documentation has been updated.*

N-02 Floating Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled. While pull request #28 fixes the pragma in `DollarStore.sol`, `SpokeLifecycleLib.sol`, and `SpokeShareLib.sol`, the in-scope files `IDollarStore.sol` and `SwapRouteLib.sol`, as well as the remaining files in the codebase, still use `pragma solidity ^0.8.24`.

Consider using fixed pragma directives consistently across the codebase.

Update: *Resolved in pull request #30*

The team stated:

> *Fixed. Using fixed pragma directives consistently across the codebase.*

N-03 Misleading Documentation of `processQueue` Return Value

The NatSpec of the `processQueue` function documents `positionsProcessed` as the number of positions filled or partially filled. However, `settleSameDirection` also counts positions that are parked without receiving anything. As a result, `processQueue` can return a non-zero `positionsProcessed` alongside an `amountFilled` of zero, and parked positions consume the `maxPositions` bound without this being documented.

Consider documenting `positionsProcessed` as the number of positions visited, including parked ones, and stating that parked positions count toward `maxPositions`.

Update: *Resolved in pull request #43 at commit 3d2ee5b.*

The team stated:

> *Fixed. The processQueue documentation now states that positionsProcessed counts every position visited, including one parked because its payout failed, and that parked positions count toward maxPositions.*

N-04 `getMinimumOrderSize` Contradicts Its Documentation for the Parked Line

The NatSpec of `getMinimumOrderSize` states that the base minimum is returned if either asset is not listed. However, for `wantAsset == address(0)`, which can never be listed, `_routeLine` resolves the parked line before checking whether the assets are listed. As a result, the function returns a minimum that grows tenfold for every 25 parked positions, and reverts with an overflow panic once the parked line holds 1,725 positions.

Consider returning the base minimum when `wantAsset` is the zero address, or documenting the parked-line behavior in the NatSpec.

Update: *Resolved in pull request #42 at commit c252822.*

The team stated:

> *Fixed. getMinimumOrderSize now returns the base minimum when wantAsset is the zero address, matching its documentation. The parked line accepts no orders, and no order was ever checked against the previous value.*

Conclusion

The audited changes remove the Guardian's ability to force-cancel queue positions and introduce a recovery path for fully impaired spokes, allowing governance to retire a pool whose outstanding receipt shares are no longer backed and to relist its asset under a fresh `poolId`. No critical- or high-severity issues were identified.

Two medium-severity issues were reported: one concerning the ability of any user to block the retirement of a winding-down spoke now that queue positions can only be closed by their owners, and one concerning escrow haircuts being applied repeatedly to the queues of relisted assets, which predates the audited changes but becomes more relevant under the new relisting-based recovery path. Two note-severity issues were also reported regarding outdated documentation and floating pragma directives.

Following the initial audit, the DollarStore team submitted pull requests #32 and #33 in response to the two medium-severity issues, replacing the conversion of escrow whose transfer fails with a parking mechanism and scoping queue keys by pool. These changes address both issues. However, one medium-severity issue was reported concerning the ability of a frozen wallet to accumulate parked positions and block other users' fills and deposits, two low-severity issues were reported concerning the gas cost of escrow haircuts over parked positions and the relisting of an asset whose escrow has not yet been haircut, and two note-severity issues were reported regarding outdated documentation.

We thank the DollarStore team for their participation during this audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.